



DOM Manipulation

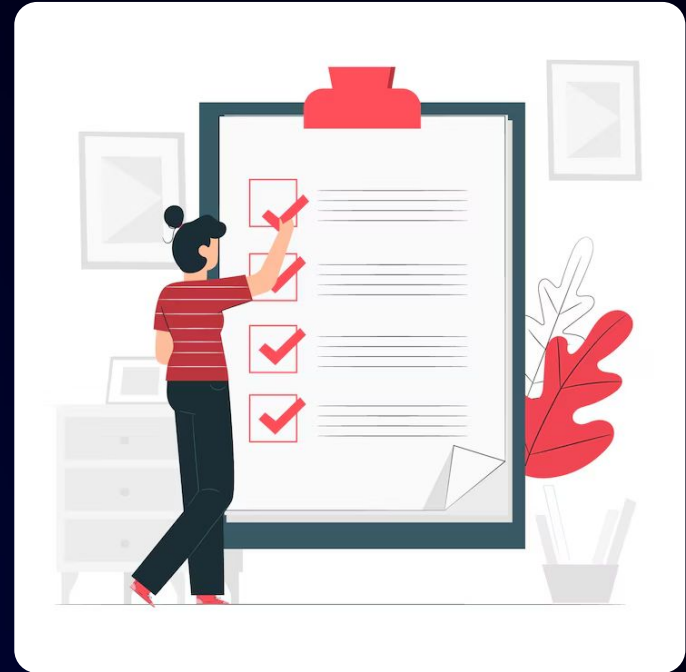
Learning Objectives

- Visualize and understand the structure and working mechanism of the DOM.
- Learn essential DOM methods for selecting and accessing HTML elements.
- Explore internal HTML object collections and their role in the DOM.
- Manipulate content and attributes of DOM elements using intermediate DOM methods.
- Create and manipulate DOM elements programmatically
- Apply advanced styling techniques and manage classes using the DOM.



Prerequisites

- Basic understanding of HTML elements, tags, and page structure.
- Fundamental knowledge of CSS selectors, classes, and styling concepts.
- Familiarity with JavaScript basics such as variables, functions, loops, and conditions.
- Understanding of JavaScript events like click and input events.
- Basic knowledge of browser developer tools and webpage inspection.





Visualization of the DOM and Its Working

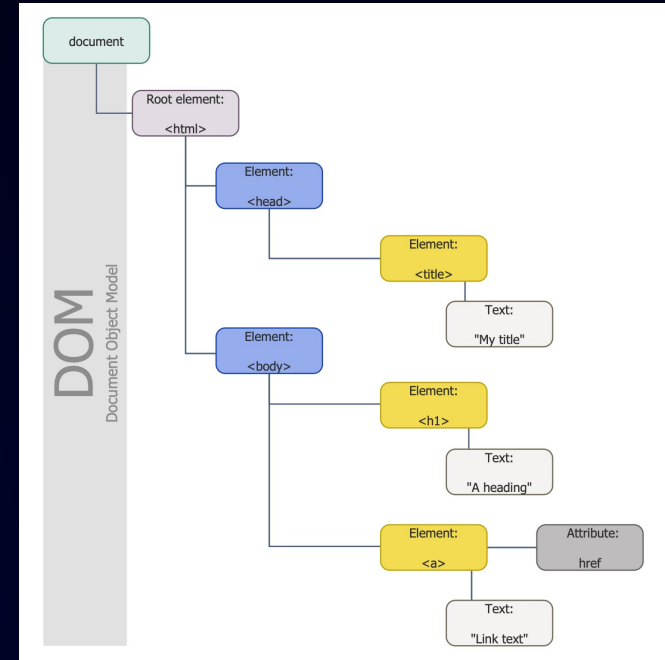
DOM (Document Object Model)

- **Definition:** The Document Object Model (DOM) is an API for HTML and XML documents. It provides a structured representation of a document as a tree of objects.

- **Purpose:** It allows programming languages to interact with the document structure, style, and content.

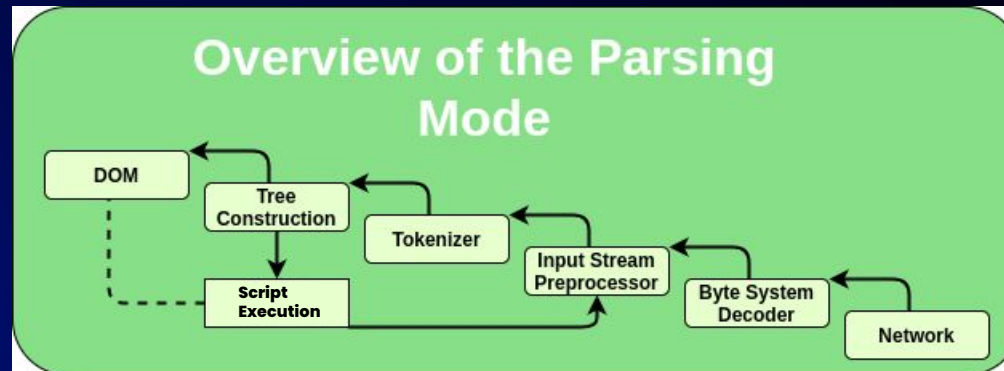
- **Components:**

- **Nodes:** The building blocks of the DOM. Includes elements, attributes, and text.
- **Tree Structure:** Represents HTML as a hierarchical tree where each node is an object.



How the DOM Works

- **HTML Parsing:** Browsers read HTML markup and convert it into a structured format.
- **DOM Construction:** The browser constructs the DOM tree by creating nodes for elements, attributes, and text.
- **Rendering:** The DOM tree is then used to render the visual representation of the webpage.
- **Updates:** Dynamic changes in the DOM reflect immediately on the webpage.



Pop Quiz

Q.What does DOM stand for?

A

Document Object Model

B

Digital Object Model

Pop Quiz

Q.What does DOM stand for?

A

Document Object Model

B

Digital Object Model



DOM Basic Methods

Essential DOM Methods

- **getElementById()**

- **Purpose:** Selects a single element with the specified ID.
- **Syntax:** `document.getElementById('elementId')`
- **Example:** `let myElement = document.getElementById('header')`

- **getElementsByClassName()**

- **Purpose:** Selects all elements with the specified class name.
- **Syntax:** `document.getElementsByClassName('className')`
- **Example:** `let items = document.getElementsByClassName('menu-item')`

Essential DOM Methods

- **getElementsByTagName()**

- **Purpose:** Selects all elements with the specified tag name.
- **Syntax:** `document.getElementsByTagName('tagName')`
- **Example:** `let paragraphs = document.getElementsByTagName('p')`

- **querySelector()**

- **Purpose:** Selects the first element that matches a specified CSS selector.
- **Syntax:** `document.querySelector('selector')`
- **Example:** `let firstItem = document.querySelector('.list-item')`

Essential DOM Methods

- **querySelectorAll()**
 - **Purpose:** Selects all elements that match a specified CSS selector.
 - **Syntax:** `document.querySelectorAll('selector')`
 - **Example:** `let allItems = document.querySelectorAll('li')`

Hands-On | Working with DOM Selectors

Pop Quiz

Q.Which method returns a NodeList of all elements matching a CSS selector?

A

`getElementById()`

B

`querySelectorAll()`

Pop Quiz

Q.Which method returns a NodeList of all elements matching a CSS selector?

A

getElementById()

B

querySelectorAll()



Understanding Internal HTML Object Collections

HTML Object Collections

- **HTMLCollection:**

- **Definition:** A live collection of HTML elements.
- **Characteristics:** Automatically updates when the document changes.
- **Usage:** Often returned by methods like `getElementsByClassName()` and `getElementsByTagName()`.

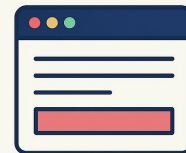
- **NodeList:**

- **Definition:** A collection of nodes (could be elements or other node types).
- **Characteristics:** Can be live or static. Static `NodeList` doesn't update with document changes.
- **Usage:** Returned by methods like `querySelectorAll()` and `childNodes`.

HTML Object Collections

HTMLCollection	NodeList
Contains only HTML elements	Can contain elements, text nodes, comments
Live collection	Can be live or static
Returned by methods like <code>getElementsByClassName()</code>	Returned by <code>querySelectorAll()</code>

HTMLCollection vs NodeList



HTMLCollection



NodeList

HTML Object Collections example

```
<button class="btn">First button</button>
<button class="btn">Second button</button>
<button class="btn">Third button</button>
```

In this example, we have three button elements. Each has a class of btn. Now, let's select the buttons using the `getElementsByClassName` method.

```
const buttonElements = document.getElementsByClassName('btn')
console.log(buttonElements)
```

```
► HTMLCollection(3) [button.btn, button.btn, button.btn]
```

Hands-On | Understanding Internal HTML Object Collections

Pop Quiz

Q.Is an HTMLCollection live or static?

A

live collection

B

Static collection

Pop Quiz

Q. Is an HTMLCollection live or static?

A

live collection

B

Static collection

HTMLCollection is a **live collection**, meaning it automatically updates when elements are added or removed from the DOM.



Intermediate DOM Methods

innerText

- Sets or retrieves the text content of an element, excluding HTML tags.
- **Usage**

```
element.innerText = "New text content";  
var text = element.innerText;
```


- Sets or retrieves the HTML content of an element, including HTML tags.
- **Usage**

```
element.innerHTML = "<b>Bold text</b>";  
var html = element.innerHTML;
```

textContent

- Sets or retrieves the text content of an element, including hidden elements.
- **Usage**

```
element.textContent = "New text content";  
var text = element.textContent;
```

setAttribute()

- Sets the value of an attribute on an element.
- **Usage**

```
element.setAttribute("data-custom", "value");
```

getAttribute()

- Retrieves the value of an attribute from an element.
- **Usage**

```
var value = element.getAttribute("data-custom");
```

Hands-On | Intermediate DOM Methods

Pop Quiz

Q. What happens if the attribute already exists?

A

It updates (overwrites) the existing value.

B

The existing value does not get overwritten.

Pop Quiz

Q. What happens if the attribute already exists?

A

It updates (overwrites) the existing value.

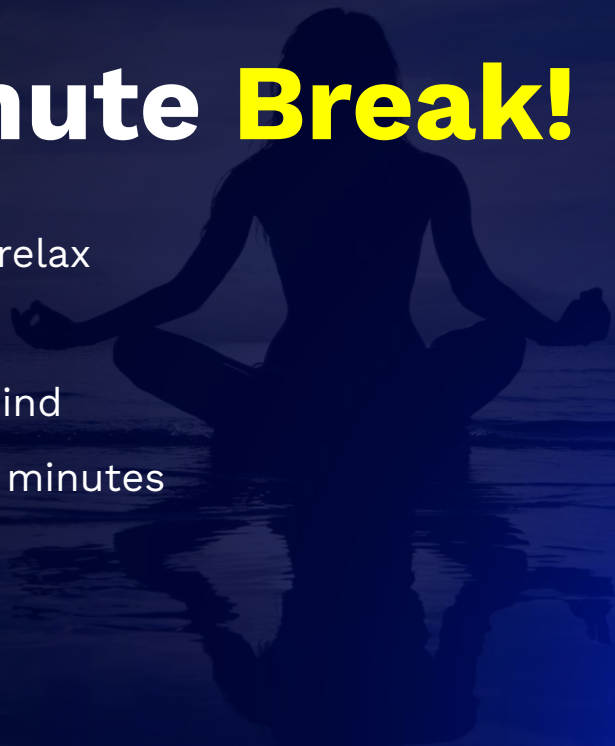
B

The existing value does not get overwritten.



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes





Element Creation and Manipulation

Creating and Manipulating DOM Elements

- **createElement()**

- **Purpose:** Creates a new HTML element.
- **Syntax:** `document.createElement('tagName')`
- **Example:** `let newDiv = document.createElement('div')`

- **appendChild()**

- **Purpose:** Adds a new child node to a parent node.
- **Syntax:** `parentNode.appendChild(childNode)`
- **Example:** `document.body.appendChild(newDiv)`

```
// Create element:  
const para = document.createElement("p");  
para.innerText = "This is a paragraph."  
  
// Append to body:  
document.body.appendChild(para);
```

Creating and Manipulating DOM Elements

- **append()**

- **Purpose:** Adds new nodes or text to the end of a parent node.
- **Syntax:** parentNode.append(node1, node2, ...)
- **Example:** parent.append('New text', newElement)

```
let div = document.createElement("div");  
let p = document.createElement("p");  
div.append("Some text", p);
```

- **removeChild()**

- **Purpose:** Removes a child node from a parent node.
- **Syntax:** parentNode.removeChild(childNode)
- **Example:** parent.removeChild(childToRemove)

```
const list = document.getElementById("myList");  
list.removeChild(list.firstChild);
```

Creating and Manipulating DOM Elements

- **replaceChild()**

- **Purpose:** Replaces one child node with another.
- **Syntax:** parentNode.replaceChild(newChild, oldChild)
- **Example:** parent.replaceChild(newElement, oldElement)

```
// Select first child element:  
const element = document.getElementById("myList").children[0];  
  
// Create a new text node:  
const newNode = document.createTextNode("Water");  
  
// Replace the text node:  
element.replaceChild(newNode, element.childNodes[0]);
```

Hands-On | Element Creation and Manipulation

Q.What is the correct order of DOM element creation and insertion?

A

- 1. Create element → createElement()**
- 2. Add content/attributes → textContent, setAttribute()**
- 3. Attach to DOM → appendChild() / append()**

B

- 1. Create element → createElement()**
- 2. Attach to DOM → appendChild() / append()**
- 3. Add content/attributes → textContent, setAttribute()**

Q.What is the correct order of DOM element creation and insertion?

A

- 1. Create element → createElement()**
- 2. Add content/attributes → textContent, setAttribute()**
- 3. Attach to DOM → appendChild() / append()**

B

- 1. Create element → createElement()**
- 2. Attach to DOM → appendChild() / append()**
- 3. Add content/attributes → textContent, setAttribute()**



DOM Styling and Classes

style Property

- Directly sets inline styles on an element.
- **Usage**

```
element.style.color = "red";
```

classList.add()

- **Definition:** Adds one or more classes to an element.
- **Usage**

```
element.classList.add("new-class");
```

classList.remove()

- **Definition:** Removes one or more classes from an element.
- **Usage**

```
element.classList.remove("old-class");
```

classList.toggle()

- **Definition:** Toggles a class on or off for an element.
- **Usage**

```
element.classList.toggle("active");
```

classList.contains()

- **Definition:** Checks if an element contains a specific class.
- **Usage**

```
var hasClass = element.classList.contains("active");
```

Practical Examples

- **Creating a Class**
- **CSS**

```
<div id="box" class="box"></div>
```

- **JavaScript**

```
var box = document.getElementById("box");  
box.classList.add("highlight");
```

Q. What method is used to change the src attribute of an image element?

A

innerHTML

B

setAttribute()

Pop Quiz

Q. What method is used to change the src attribute of an image element?

A

innerHTML

B

setAttribute()



Guided Practice Tasks on **DOM Manipulation Deep Dive**

Summary

In this lecture, we have covered:

- The DOM represents an HTML page as a tree structure that JavaScript can access and modify.
- Elements are selected using `getElementById`, `querySelector`, and `querySelectorAll`.
- Content can be changed using `innerText`, `textContent`, and `innerHTML`.
- Attributes are handled using `setAttribute()` and `getAttribute()`.
- New elements are created with `createElement()` and added/removed using `append()` and `removeChild()`.
- CSS classes are managed using `classList`, and styles are changed using the `style` property.

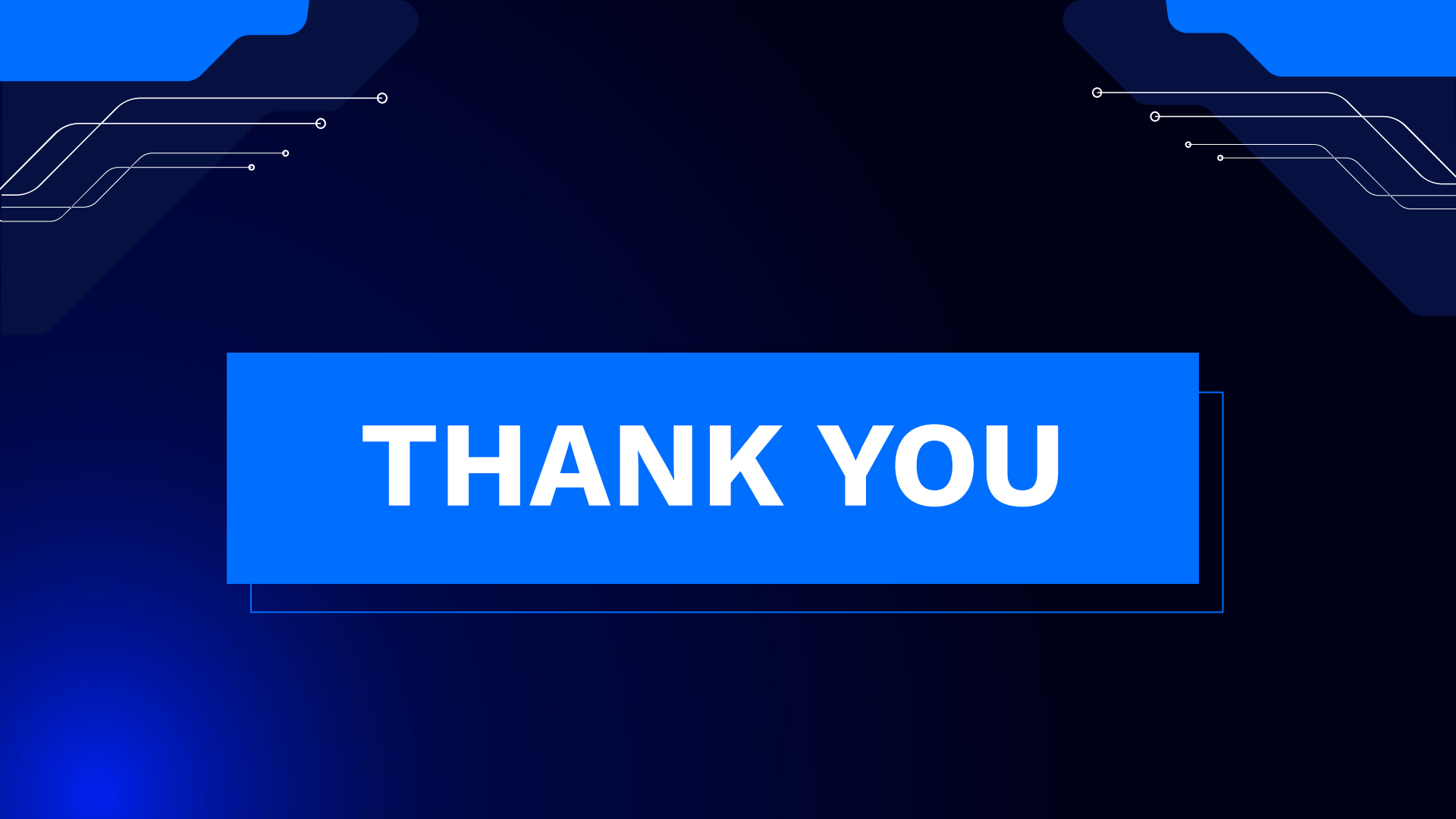




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU